

GAME DESIGN DOCUMENT

Dự án: Neon Survivor - Dành cho Unity Developer (2 YOE)

1. Tổng quan dự án (Project Overview)

- Tên game:** Neon Survivor
- Thể loại:** Casual, Action Roguelike, Bullet Heaven (Wave Survival).
- Nền tảng:** PC (Windows/macOS).
- Góc nhìn:** Top-down 2D.
- Core Loop (Vòng lặp cốt lõi):** Di chuyển né tránh → Tự động tấn công → Tiêu diệt quái → Nhận EXP → Lên cấp → Chọn kỹ năng ngẫu nhiên → Sống sót qua các Wave.

2. Gameplay Mechanics (Cơ chế Game)

2.1. Nhân vật chính (Player)

- Điều khiển:** Sử dụng W, A, S, D hoặc phím mũi tên để di chuyển (Áp dụng Unity New Input System).
- Tấn công:** Tự động xả skill/đạn vào kẻ địch gần nhất hoặc theo hướng nhân vật đang nhìn.
- Chỉ số cơ bản:** HP (Máu), MvSpd (Tốc độ chạy), AtkSpd (Tốc độ đánh), Dmg (Sát thương).

2.2. Kẻ địch & Hệ thống Wave (Enemies & Waves)

- Map:** Một map duy nhất vô tận (hoặc giới hạn viền), không có chướng ngại vật phức tạp để tập trung vào số lượng quái.
- Wave System:** Mỗi wave kéo dài 60 giây. Cứ sau 60 giây quái sẽ mạnh hơn, đổi loại quái hoặc xuất hiện Boss. Số lượng quái trên màn hình có thể lên tới 200 - 500 con cùng lúc.

2.3. Hệ thống Kỹ năng & Nâng cấp (Roguelike Elements)

- Khi tiêu diệt quái, chúng rớt ra các viên EXP.
- Nhặt đủ EXP, Player lên cấp. Game tạm dừng (`Time.timeScale = 0`).
- Hiển thị UI: Chọn 1 trong 3 kỹ năng/chỉ số ngẫu nhiên (Vd: Bắn thêm đạn, Tăng máu, Gọi sét...).

3. Kiến trúc kỹ thuật (Technical Architecture)

Yêu cầu thể hiện kỹ năng của Unity Dev 2 năm kinh nghiệm: Kiến trúc code sạch, Scalability và Performance Optimization.

3.1. Design Patterns & Architecture

- **MVC / MVP Pattern:** Tách biệt rõ ràng Data khỏi UI.
- **Observer Pattern (Event/Delegate/Action):** Sử dụng event để xử lý hệ thống UI thay vì update mỗi frame (Ví dụ: OnHealthChanged).
- **State Pattern (FSM):** Quản lý trạng thái Game (MainMenuState, PlayingState, v.v.) và AI của Boss (Chasing, Attacking, SkillCast).
- **Factory Pattern:** Kết hợp Object Pool để sinh quái vật và kỹ năng theo ID.

3.2. Data Management (Quản lý Dữ liệu)

- **Scriptable Objects (SO):** Lưu trữ toàn bộ chỉ số Game, Enemy Data, Skill Data giúp cân bằng game dễ dàng.
- **Save/Load System:** Lưu tiến trình bằng JSON thông qua JsonUtility hoặc Newtonsoft.Json bất đồng bộ.

3.3. Core Systems & Tối ưu hóa (Optimization)

- **Object Pooling (Bắt buộc):** Xây dựng ObjectPoolManager generic để tái sử dụng Đạn, Quái vật, EXP, VFX. Tuyệt đối không Instantiate/Destroy liên tục.
- **Spatial Partitioning / Đơn giản hóa Vật lý:** Dùng CircleCollider2D cơ bản hoặc toán học (Physics2D.OverlapCircleNonAlloc, Vector3.Distance) thay vì Rigidbody+Collider phức tạp.
- **Garbage Collection (GC) Control:** Tránh khởi tạo biến mới trong Update(), hạn chế LINQ trong loop, dùng StringBuilder cho UI Text.
- **Addressables:** Load/unload prefab, âm thanh, VFX bất đồng bộ để giảm tải RAM.

3.4. UI & Animation

- **UGUI Optimization:** Chia Canvas động và tĩnh. Tắt Raycast Target ở các element không tương tác.
- **Animation Event:** Căn frame gọi hàm gây sát thương chính xác theo Animation.

4. Lộ trình phát triển (Roadmap / Milestones)

Mốc 1: Khởi tạo Kiến trúc & Core Player

- **Thiết lập:** Project 2D URP, cấu trúc thư mục chuẩn.
- **Input:** Tích hợp Unity New Input System, lập trình di chuyển Top-down mượt mà.
- **Data Architecture:** Tạo các Scriptable Object cơ bản (PlayerStatsSO, GameSettingsSO).
- **State Machine:** Khởi tạo Game Manager dùng FSM (MainMenu, Playing).

Mốc 2: Hệ thống Tối ưu & Kẻ địch (Enemies)

- **Object Pooling:** Hoàn thiện ObjectPoolManager generic cho Quái, Đạn, VFX.
- **Enemy AI:** Tìm và di chuyển tới Player (Tối ưu: cập nhật vị trí Player 0.2s/lần thay vì mỗi frame).
- **Wave Spawner:** Tạo WaveManager kết hợp WaveDataSO để sinh quái ngoài tầm nhìn camera.

Mốc 3: Hệ thống Chiến đấu & Vật lý

- **Vũ khí:** Hệ thống Auto-Aim quét quái gần nhất và xả đạn (từ Pool).
- **Tối ưu Vật lý:** Thay thế OnCollisionEnter2D bằng Physics2D.OverlapCircleNonAlloc để tính sát thương.
- **Sức khỏe:** Class HealthComponent dùng chung, kết nối Observer Pattern (OnDeath) trả object về Pool.

Mốc 4: Vòng lặp Roguelike (EXP & Skills)

- **Kinh nghiệm (EXP):** Rút viên EXP từ Pool, Player có khả năng hút EXP.
- **Lên cấp:** Khi đầy EXP → OnLevelUp → Chuyển sang LevelUpState (Tạm dừng game).
- **Roguelike Skills:** Thuật toán lấy ngẫu nhiên 3 kỹ năng từ kho SkillSO, áp dụng trực tiếp vào Player Stats.



Mốc 5: Giao diện (UI) & Lưu trữ Dữ liệu

- **UI Architecture:** Gắn Event cập nhật Health Bar, Timer, Level (MVC/MVP). Chia Static và Dynamic Canvas.
- **Hệ thống Save/Load:** Lưu High Score, Vàng qua file JSON bất đồng bộ.



Mốc 6: Đánh bóng, Addressables & Hoàn thiện

- **Asset Management:** Cấu hình Addressables cho các Prefab nặng.
- **Sound Manager:** Hệ thống Audio Pool cho SFX số lượng lớn.
- **Tooling:** Viết Custom Editor (Inspector) để thiết kế Wave dễ dàng.
- **Profiling & Build:** Dùng Unity Profiler xử lý Memory Leak, GC spikes và xuất bản build .exe.